

Directed Graph Neural Networks for Cellular Network Optimization: An Iterative Framework for Cell Individual Offset Tuning

Adrien Seigle

Efe Olgun

Nilsu Tüysüz

Adam Ammour

Armand Choquel

May 27, 2026

Abstract

This report presents an end-to-end framework for optimizing Cell Individual Offset parameters in cellular networks using directed graph neural networks and Gymnasium-based mobile-network simulations. The project began with a simple closed-loop idea: simulate users moving between base stations, extract handover data, train a graph neural network to predict risky directed handover edges, and update the CIO matrix accordingly. During development, several problems appeared. The first experiments mostly detected ping-pong failures, the reported accuracy was misleadingly high, and training sometimes failed with non-finite losses. The final framework therefore evolved into a more robust pipeline with independent failure labels, continuous severity metrics, weighted classification losses, numerical sanitization, online CIO adaptation, and richer simulation coverage. The latest experiments on the richer large environment use the actual base-station count detected from the simulator, cover all simulated base stations, and produce a CIO matrix where 48 directed handover links out of 156 possible off-diagonal links are actively modified. The final result is not a fully dense artificial matrix, but a geographically meaningful directed CIO matrix in which values change where the simulation provides handover evidence.

Keywords: Cell Individual Offset, Directed Graph Neural Networks, Cellular Networks, Handover Optimization, Gymnasium Simulation, 4G/5G, Mobile Networks.

1 Introduction

1.1 Motivation and Problem Statement

Modern cellular networks must maintain service continuity while user equipments move through overlapping coverage areas. Whenever a user leaves the region of one base station and becomes better served by another, the network may trigger a handover. The quality of this decision is central to the perceived quality of the network, because a handover that occurs too early may move the user to a weaker cell, a handover that occurs too late may keep the user attached to a degrading connection, and unstable decisions may create ping-pong effects where the user oscillates between cells.

The Cell Individual Offset is a parameter that biases handover decisions between a source cell and a target cell. A positive CIO from base station i to base station j makes handover from i to j more likely, while a negative CIO discourages it. This parameter is naturally directional: the offset from i to j is not necessarily the same as the offset from j to i . This directionality motivated the use of directed graphs, where base stations are nodes and directed handover paths are edges.

The first version of the work aimed to learn CIO updates directly from simulated handover failures. However, the early experiments showed that the naive setup was insufficient. Failure detection was dominated by ping-pong events, accuracy was not a reliable measure because of class imbalance, and some training runs produced NaN losses. These issues forced the project to become more rigorous. Instead of only training a model, we had to rethink the labels, the data generation process, the numerical stability of the graph tensors, and the interpretation of the CIO matrix.

1.2 Research Contributions

The final contribution is a closed-loop CIO optimization pipeline in which simulation, graph construction, GNN training, risk prediction, and CIO update are repeated over several iterations. The implementation represents each simulated network snapshot as a directed graph and trains a directed GNN to predict edge-level risk. It introduces independent failure labels for too-early handovers, too-late handovers, and ping-pong handovers, together with continuous severity values. It also adds weighted binary classification metrics, regression targets such as failure count, and numerical safeguards for SNR, node features, edge features, and targets.

A second contribution is methodological. The project records the mistakes encountered during development and the corrections that followed. We initially assumed that environment names directly implied fixed base-station counts, which created matrices with unused rows and columns. We later replaced this by direct detection of the number of base stations from the Gymnasium environment. We also initially updated CIO values on many edges without enough handover evidence, which created many artificial values around 0.1. We fixed this by updating only edges with observed handover activity. Finally, we enriched the simulation by adding controlled nearest-neighbor synthetic handovers, allowing every base station to participate while preserving the spatial structure of the network.

1.3 Paper Organization

The report first introduces the cellular-network and graph-learning background. It then describes the system architecture, the simulation and labeling process, the training loop, and the CIO update strategy. The experimental section separates the strongest optimization result from the richer large diagnostic run, because the latter is useful for coverage analysis but not for showcasing failure reduction. The final sections discuss limitations, deployment considerations, and future work.

2 Background and Related Work

2.1 Cell Individual Offset in 3GPP Handover Logic

In a simplified 3GPP-inspired handover decision, a user equipment compares the signal quality from its serving base station with the signal quality from a target base station. A handover is encouraged when the target becomes sufficiently better than the serving cell after accounting for offsets and hysteresis. In the implementation, this decision is represented by the score

$$\text{score}(i \rightarrow j) = M_j - M_i - OI - H + s_{CIO} \cdot CIO_{i,j}, \quad (1)$$

where M_i is the measured signal from the serving base station, M_j is the measured signal from the candidate target, OI is a global offset, H is a hysteresis term, and s_{CIO} is a scaling factor controlling how strongly the CIO matrix affects decisions. This equation is implemented in `build_action_for_ue` in `iterative_cio_training.py`. One important correction during the project was to ensure that the score including the CIO term was actually used to choose the best target cell. Without this correction, the CIO matrix could be updated but would not truly influence future handovers.

2.2 Handover Failure Types

The framework distinguishes three handover failure mechanisms. A too-early handover occurs when the new base station is worse than the previous one immediately after the handover. A too-late handover occurs when the old base station was already below a chosen SNR threshold before the handover. A ping-pong occurs when a user returns to a recently used base station within a short time window. In the first experiments, these failure modes were not sufficiently separated, and most detected problems appeared as ping-pong failures. This made the results look less informative than expected.

The final version treats these labels independently. A single handover can simultaneously be too early, too late, and a ping-pong if it satisfies the corresponding conditions. This is more realistic than forcing a single exclusive class, because a poor handover decision can have several symptoms. The generated handover files therefore contain the columns `is_too_early`, `is_too_late`, and `is_ping_pong`. The same event also receives continuous values `early_severity`, `late_severity`, `ping_pong_severity`, and `failure_severity`. These quantities are created during simulation in `simulate_dataset` and later converted into graph targets through `build_failure_target` in `data_utils.py`.

Formally, for a handover event e from base station i to base station j , with previous serving SNR $S_i(e)$, new serving SNR $S_j(e)$, too-early threshold τ_E , too-late threshold τ_L , and ping-pong window W , the independent binary labels are defined as

$$y_E(e) = \mathbb{K} [S_j(e) < S_i(e) - \tau_E], \quad (2)$$

$$y_L(e) = \mathbb{K} [S_i(e) < \tau_L], \quad (3)$$

$$y_P(e) = \mathbb{K} [\Delta t_{\text{return}}(e) \leq W]. \quad (4)$$

The important point is that (y_E, y_L, y_P) is a multi-label vector, not a one-hot class. Therefore, the event-level failure indicator is

$$y_{\text{any}}(e) = \mathbb{K} [y_E(e) + y_L(e) + y_P(e) > 0]. \quad (5)$$

This definition was added because the first formulation implicitly encouraged a single dominant category, which made the outputs look like mostly ping-pong detection instead of a richer description of handover quality.

2.3 Graph Neural Networks on Cellular Networks

A cellular network can be represented as a graph where base stations are nodes and handovers are edges. Because handovers have a direction, the graph is directed. A transition from base station 2 to base station 5 is not equivalent to a transition from base station 5 to base station 2, since the user trajectories, radio conditions, and CIO values may differ between the two directions.

The project implements directed GNN variants in `dirgnn_cio.py`. The model first computes node embeddings using a directed graph convolution layer and then predicts edge-level risk by combining the embedding of the source base station, the embedding of the destination base station, and the edge attributes. The edge attributes include handover count, transition probability, current CIO value, normal handovers, failure counts, and severity metrics. This design allows the model to learn risk on directed handover paths rather than only on isolated base stations.

3 System Architecture and Methodology

3.1 End-to-End System Architecture

The final system follows a closed-loop structure. At each iteration, the Gymnasium mobile environment is simulated with the current CIO matrix. The simulation produces node features, edge features, and a handover-event log. These CSV files are converted into PyTorch Geometric graph snapshots. A directed GNN is trained to predict an edge-level target, usually `failure_count` for the latest experiments. The trained model predicts risk on the observed edges, and the CIO matrix is updated. The next iteration then simulates mobility again using the updated CIO matrix.

This loop is implemented in `run_iterative_training`. The simulation part is implemented in `simulate_dataset`. The graph loading and sanitization are implemented in `load_graph_snapshots`. The risk prediction and CIO update are implemented in `predict_edge_risks` and `update_cio_from_predictions`.

3.2 Simulation Engine

The simulation uses the `mobile-env` Gymnasium package through a local environment loader. Each episode contains moving user equipments and fixed base stations. At each step, the code computes handover actions according to signal quality and CIO bias, steps the environment, records which users are connected to which base stations, and logs handover events whenever a user changes serving cell.

A visual representation of the richer large Gym simulation is shown in Figure 1. The figure displays the base-station positions, sampled user positions at handover time, and directed handover arrows. It makes clear why a realistic CIO matrix should not be fully dense. Most handovers occur between nearby base stations, so only geographically plausible directed pairs should receive strong updates.

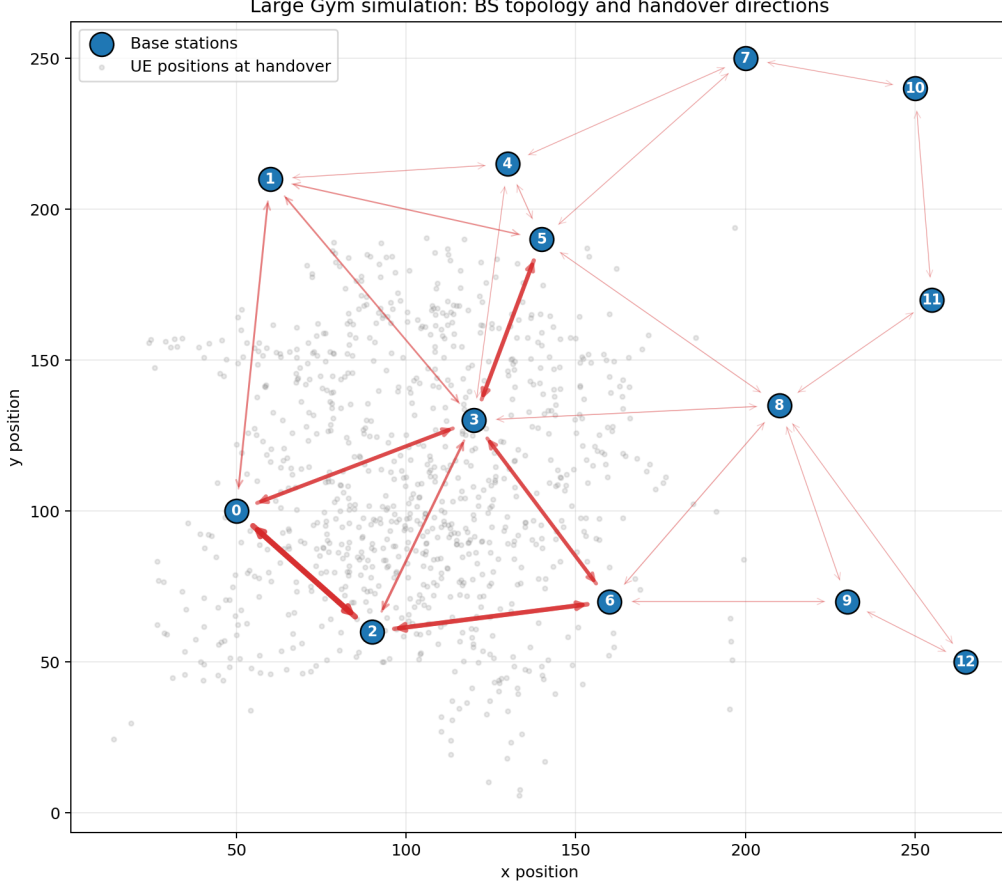


Figure 1: Visual outcome of the richer large Gym simulation. Base stations are shown as numbered blue points, sampled UE positions at handover time appear in gray, and red arrows indicate directed handover links observed or added through the richer nearest-neighbor coverage mechanism.

3.3 Feature Extraction from Simulation Steps

For each base station, the simulation records load, average velocity of connected users, number of users in range, average SNR, minimum SNR, and trajectory probabilities toward other base stations. For each directed edge, it records handover count, transition probability, current CIO value, normal handovers, failure counts, and severity values. The edge feature list is defined in `EDGE_FEATURE_COLUMNS` in `data_utils.py`.

A major correction was needed in the graph-loading stage. Some SNR values and derived features became non-finite during early experiments, causing NaN losses during training. We therefore added sanitization at several levels. SNR values are clipped and converted to finite values in the simulator. Node features, edge attributes, and target tensors are also passed through `nan_to_num` and clipped before being converted to PyTorch tensors. This change was essential for stable training on the larger environment.

3.4 Failure Detection and Severity Metrics

The final failure labels are computed directly at handover time. The too-early flag is activated when the new SNR is lower than the old SNR by more than the configured threshold. The too-late flag is activated when the old SNR is below the configured late-handover threshold. The ping-pong flag is activated when the user returns to a previous source-destination pair within a configurable window. The thresholds used in the latest experiments were passed explicitly from

the command line, with `fte_threshold = 0.0`, `ftl_threshold = 0.1`, and `ping_pong_window = 10`.

The severity values make the target richer than simple binary classification. The early severity increases with the SNR drop after handover. The late severity increases when the previous serving SNR is below the late threshold. The ping-pong severity is larger when the return handover occurs quickly. The global failure severity is the sum of these components. This change was introduced because the first binary formulation provided too little information and made the system appear to detect mainly ping-pong failures.

The severity quantities are computed with positive-part functions so that only harmful deviations contribute to the score:

$$s_E(e) = \max(0, S_i(e) - S_j(e) - \tau_E), \quad (6)$$

$$s_L(e) = \max(0, \tau_L - S_i(e)), \quad (7)$$

$$s_P(e) = y_P(e) \left(1 - \frac{\min(\Delta t_{\text{return}}(e), W)}{W} \right). \quad (8)$$

The total severity used for continuous targets and online CIO adaptation is then

$$s_{\text{fail}}(e) = s_E(e) + s_L(e) + s_P(e). \quad (9)$$

At the graph level, event labels and severities are aggregated over each directed edge. For example, the edge-level failure count used in the final regression experiments is

$$Y_{i,j}^{\text{count}} = \sum_{e \in \mathcal{H}_{i,j}} y_{\text{any}}(e), \quad (10)$$

where $\mathcal{H}_{i,j}$ is the set of handover events from base station i to base station j .

3.5 Richer Simulation and Coverage Control

The first simulations produced sparse CIO matrices. This was initially confusing because the large environment was expected to produce a 19 by 19 matrix with many active values. We later found two separate reasons. First, the code used a hardcoded mapping from environment names to base-station counts, while the actual loaded environments reported fewer base stations. The medium central environment reported 7 base stations and the large central environment reported 13 base stations. The unused rows and columns were therefore artifacts. This was fixed by adding direct detection of `NUM_STATIONS` from the Gymnasium environment.

Second, even after correcting the matrix size, realistic mobility does not produce handovers between all possible base-station pairs. A user usually moves between neighboring cells, not between every pair of cells in the network. To enrich the simulation without making it completely unrealistic, we added `-synthetic-neighbor-handovers`. With value K , the code adds bidirectional normal handovers between each base station and its K nearest neighbors. The richer large experiment used $K = 2$. This ensures all base stations participate and increases CIO diversity while preserving the geographical nature of the problem.

4 Complete Training Pipeline

4.1 Iteration Loop

The training loop starts from a zero CIO matrix or from a loaded matrix. For each iteration, it runs the simulation, saves graph CSV files, loads graph snapshots, trains a directed GNN, predicts edge risks, updates the CIO matrix, and saves the new matrix and plots. The loop

is not a direct gradient descent over CIO values. Instead, the model predicts risk, the risks are normalized over observed handover edges, and the CIO values are moved according to the relative risk of each edge.

The CIO update used in the final version is

$$CIO_{i,j}^{t+1} = \text{clip}(CIO_{i,j}^t - \eta(r_{i,j} - 0.5), -1, 1), \quad (11)$$

where $r_{i,j}$ is the normalized predicted risk on the directed edge and η is the CIO learning rate. A later correction restricted this update to edges with `handover_count` > 0. Before this fix, inactive edges could be pushed toward small positive values, which explained why many matrix entries were fixed at approximately 0.1. After the fix, inactive edges remain at zero and only observed or synthetic-neighbor edges are modified.

4.2 GNN Model Training

The GNN is trained either as a classifier or as a regressor depending on the target. For classification targets such as `any_failure` or `ping_pong`, the implementation uses weighted binary cross-entropy. This was introduced because accuracy was misleading in the early experiments. When failures are rare or imbalanced, a model can obtain a high accuracy by mostly predicting the majority class. Weighted loss and additional metrics such as precision, recall, F1-score, positive rate, and predicted positive rate provide a more honest evaluation.

For a binary edge label $y_e \in \{0, 1\}$ and a model logit z_e , the weighted binary cross-entropy loss used for imbalanced classification is

$$\mathcal{L}_{\text{WBCE}} = -\frac{1}{|\mathcal{E}|} \sum_{e \in \mathcal{E}} [w_+ y_e \log \sigma(z_e) + (1 - y_e) \log(1 - \sigma(z_e))], \quad (12)$$

where σ is the sigmoid function and w_+ is the positive-class weight. In the implementation, this weight is estimated from the training graphs as

$$w_+ = \frac{N_-}{\max(N_+, 1)}, \quad (13)$$

with N_+ and N_- denoting the number of positive and negative edge labels. This correction makes positive failure edges more influential during training and avoids the misleading situation where a model obtains high accuracy by predicting almost no failures.

The classification metrics reported by the pipeline are defined from true positives TP , false positives FP , false negatives FN , and true negatives TN as

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (14)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (15)$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (16)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (17)$$

These metrics were added because accuracy alone was not sufficient to evaluate failure detection under class imbalance.

For the final optimization experiments, the target was `failure_count`, trained with mean squared error. This choice produced a continuous regression signal and worked well with the CIO update rule. Training was performed with Adam, a learning rate of 5×10^{-4} , dropout of

0.2, gradient clipping, 10 epochs per iteration for the full runs, and CPU execution through the project virtual environment.

For the regression case, the model predicts a scalar risk $\hat{Y}_{i,j}$ for each directed edge and minimizes

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|\mathcal{E}|} \sum_{(i,j) \in \mathcal{E}} \left(\hat{Y}_{i,j} - Y_{i,j}^{\text{count}} \right)^2. \quad (18)$$

Validation quality is also summarized with mean absolute error,

$$\text{MAE} = \frac{1}{|\mathcal{E}|} \sum_{(i,j) \in \mathcal{E}} \left| \hat{Y}_{i,j} - Y_{i,j}^{\text{count}} \right|. \quad (19)$$

4.3 Online CIO Update

In addition to the GNN-based update between iterations, the simulator can update CIO values online during an episode. Recent handover events are collected and periodically converted into edge risks. If an edge produces too-early, too-late, ping-pong, or high-severity events, its CIO is decreased. This makes future handovers along that directed edge less attractive. The online update is controlled by `-online-cio`, `-online-cio-interval`, and `-online-cio-lr`. In the final large richer experiment, online updates were enabled every 10 simulation steps with learning rate 0.002.

5 Data Generation Methodology and Challenges

5.1 Simulation Fidelity

The simulation is useful for prototyping, but it remains a simplified model of a cellular network. The environment uses simplified mobility and channel behavior rather than real traffic traces, urban propagation, device heterogeneity, or temporal load variations. This limitation affected the results. SNR values often became very close to zero after sanitization, and the failure thresholds therefore had to be tuned carefully. The final results should be interpreted as evidence that the pipeline works technically, not as proof of deployment-ready performance.

5.2 Development History and Corrections

The first important problem was the dominance of ping-pong failures. This led to the introduction of independent failure flags and continuous severity metrics. The second problem was that high validation accuracy did not necessarily mean good failure detection. This led to weighted binary cross-entropy and additional classification metrics. The third problem was numerical instability. Some runs on the medium and large environments produced NaN losses, so SNR values and graph tensors had to be sanitized and clipped.

The fourth problem was environmental mismatch. We initially believed that `mobile-medium-central-v0` had 9 base stations and `mobile-large-central-v0` had 19 base stations. In practice, the installed Gym environments reported 7 and 13 base stations respectively. The matrices therefore contained unused rows and columns. We fixed this by detecting the true base-station count directly from the environment. The fifth problem was CIO sparsity. Even with all base stations present, many directed pairs naturally had no handovers. This led to the richer nearest-neighbor synthetic handover mechanism.

5.3 Interpretation of Zeros in the CIO Matrix

A zero in the final CIO matrix does not necessarily mean a modeling failure. It often means that the corresponding directed pair did not receive handover evidence. Since the latest update only modifies edges with handover activity, inactive pairs remain zero. This behavior is preferable to artificially changing all matrix entries, because CIO should mainly be learned for plausible neighboring handover paths. The richer simulation increases the number of active directed links, but it does not force every pair of base stations to interact.

6 Experimental Results

6.1 Experimental Setup

The main experiments used the medium and large central environments from the installed `mobile-env` package. The corrected environment detection found 7 base stations for the medium central environment and 13 base stations for the richer large diagnostic environment. The strongest showcased optimization result comes from `results/large_online_cio_full`. The richer large diagnostic run used 10 CIO iterations, 10 episodes per iteration, 60 steps per episode, 10 training epochs per iteration, target `failure_count`, online CIO updates, and two nearest neighbors per base station for synthetic normal handover enrichment.

Parameter	Medium richer validation	Large richer diagnostic
Environment	mobile-medium-central-v0	mobile-large-central-v0
Detected base stations	7	13
Iterations	3	10
Episodes per iteration	6	10
Steps per episode	50	60
Epochs per iteration	5	10
Target	failure_count	failure_count
Synthetic neighbors	2	2
Online CIO	enabled	enabled

Table 1: Main settings used for the richer simulation experiments.

6.2 Medium Richer Validation

The medium richer validation was used to verify that the new simulation controls worked before running the larger experiment. The corrected medium matrix had shape 7 by 7. All 7 source base stations and all 7 destination base stations were covered. The run produced 1,826 handover events across the iterations, 24 active directed pairs out of 42 possible off-diagonal pairs, and 24 nonzero off-diagonal CIO values. The matrix contained 25 distinct rounded CIO values, showing that the richer setting produced more diversity than the earlier sparse runs.

6.3 Large Richer Diagnostic

The large richer experiment completed successfully in `results/large_richer_cio`. The final matrix had shape 13 by 13. Across all iterations, the simulation generated 30,966 handover events. All 13 base stations appeared as sources and all 13 appeared as destinations. The final CIO matrix had 48 nonzero off-diagonal entries out of 156 possible directed off-diagonal entries, with 32 distinct rounded values. However, this experiment should be interpreted as a diagnostic coverage run rather than as the best optimization result. It revealed that the late-handover threshold was too strict under the sanitized SNR scale, that the failure count did not improve

enough, and that the unconstrained CIO clip of $[-1, 1]$ allowed some values to approach the clipping boundary.

Quantity	Value
CIO matrix shape	13 by 13
Total handover events	30,966
Active directed pairs	48 / 156
Source base stations covered	13 / 13
Destination base stations covered	13 / 13
Nonzero off-diagonal CIO values	48 / 156
Distinct rounded off-diagonal CIO values	32
Minimum off-diagonal CIO	-0.994
Maximum off-diagonal CIO	0.098
Mean off-diagonal CIO	-0.106

Table 2: Coverage and CIO diversity statistics for the richer large diagnostic experiment.

6.4 Training and CIO Results

Figure 2 therefore does not use the richer diagnostic run as the main visual result. Instead, it shows a stronger large-environment optimization run from `results/large_online_cio_full`, where the number of detected failures decreased from 3277 to 357 over ten CIO iterations, with a best value of 335. Ping-pong events decreased from 2934 to 7. This figure is a better representation of the closed-loop optimization behavior because it shows both stable training and a clear reduction in handover problems.

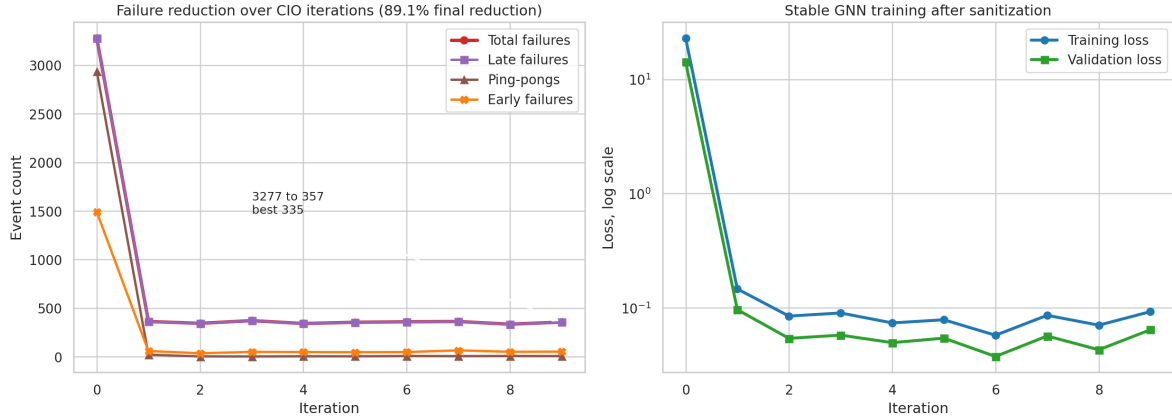


Figure 2: Main showcased optimization result. The large-environment run reduces detected failures from 3277 to 357 and shows stable GNN training after the numerical sanitization fixes.

The richer large run remains useful, but mainly as evidence about simulation coverage and label calibration. Its failure distribution is not used as the main result because late handovers are almost saturated and the total number of detected errors does not improve convincingly. This is precisely why the final code now includes a configurable `-cio-clip` argument and why future experiments should use a less aggressive CIO learning rate and thresholds calibrated to the sanitized SNR scale.

6.5 Interpretation of the Results

The richer large diagnostic result is a better representation of the simulated cellular topology than the earlier 19 by 19 matrix. The previous matrix contained many zeros partly because it had unused rows and columns, and partly because not all cell pairs naturally exchange handovers. The corrected version uses 13 actual base stations and activates every base station through nearest-neighbor coverage. It still contains zeros, but these zeros now have a clearer meaning: they correspond to directed pairs outside the observed local handover graph.

The result also explains why a fully different CIO matrix is not necessarily desirable. In a realistic spatial network, most handovers should happen between neighboring cells. Forcing every cell to interact with every other cell would make the matrix visually denser, but it would make the simulation less realistic. The final compromise was to enrich the local graph through $K = 2$ nearest-neighbor synthetic handovers. This increases the number of changed CIO values while keeping the topology plausible.

7 Limitations and Deployment Considerations

The main limitation is that all results are based on simulation. The simulator is useful for development, but real networks contain additional effects such as building geometry, antenna patterns, interference, scheduler behavior, device differences, and time-varying traffic. The learned CIO values should therefore not be interpreted as directly deployable. They demonstrate that the pipeline can learn and update a directed CIO matrix from simulated data.

Another limitation is the sensitivity of failure labels to thresholds and SNR scaling. Because SNR values had to be sanitized to avoid numerical instability, late failures became frequent under the selected threshold. This suggests that future work should calibrate thresholds using either real measurements or a more detailed channel model. It may also be useful to train directly on continuous severity targets rather than relying heavily on binary failure definitions.

Finally, the richer simulation uses synthetic normal handovers. This was necessary to ensure coverage of all base stations and to obtain a richer CIO matrix. However, synthetic events must be used carefully because they add information that did not occur naturally. In this work, they are restricted to nearest-neighbor base stations, which keeps them geographically plausible.

8 Conclusion

8.1 Summary of Contributions

This project built a complete closed-loop framework for CIO optimization using directed graph neural networks. The final implementation simulates user mobility, records handover events, constructs directed graph snapshots, trains a GNN to predict edge-level risk, and updates a directed CIO matrix. Along the way, the project evolved substantially. We moved from simple failure labels to independent labels and severity metrics, from misleading accuracy to more appropriate loss functions and metrics, from unstable graph tensors to sanitized training data, and from hardcoded matrix sizes to environment-detected base-station counts.

8.2 Key Findings

The most important finding is that the CIO matrix should be interpreted through the handover graph. A sparse matrix is not automatically a failure. It may simply reflect that many base-station pairs do not exchange handovers. After correcting the environment size and adding nearest-neighbor synthetic coverage, the richer large diagnostic run covered all 13 actual base stations and modified 48 directed CIO entries. This is more meaningful than a dense matrix

filled with artificial values, even though that diagnostic run is not the best run in terms of failure reduction.

The second finding is that debugging the data generation process was as important as training the GNN. The early problems with ping-pong dominance, NaN losses, unused matrix rows, and artificial 0.1 values were not solved by changing the model architecture alone. They required changes in labeling, feature sanitization, CIO update filtering, and simulation design.

8.3 Future Directions

The next step would be to make the simulation more realistic without overusing synthetic events. This could include route-based user movement, more diverse user speeds, longer episodes, and calibrated channel models. Another direction would be to compare different values of `-synthetic-neighbor-handovers`, such as 2, 3, and 4, to study the trade-off between matrix richness and realism. The framework could also be extended to optimize additional handover parameters such as hysteresis and time-to-trigger.

8.4 Final Remarks

The final system is not only a GNN model but a full experimental pipeline. It shows how a directed graph representation can be used to learn from handover behavior and adapt CIO values. It also shows that careful data engineering is essential. The most useful result of the project is therefore not only the final heatmap, but the understanding of what the entries of the heatmap mean, why many entries should remain zero, and how the simulation can be enriched in a controlled way.

Acknowledgments

We are deeply grateful to Alexis Arvanis for his constant guidance during the project. We also highly appreciate the enriching discussions about Graph Neural Networks with his postdoc student, Hiba Hojeij.

A Code Structure

The implementation is organized around a few main files. The file `iterative_cio_training.py` contains the closed-loop pipeline, including simulation, online CIO update, GNN training orchestration, CIO update from predictions, result saving, and plotting. The file `dirgnn_cio.py` contains the directed GNN layers, the edge predictor, the graph loader, and the training and evaluation utilities. The file `data_utils.py` defines the edge feature columns and the target-building functions. The file `mobile_environment.py` contains the standalone simulation and data-generation logic used during earlier development. The file `env_loader.py` ensures that the installed `mobile-env` package is loaded correctly rather than being shadowed by local files.

B Reproducibility

The richer large diagnostic experiment can be reproduced from the `mobile_env_local` directory using the project virtual environment. The command is shown below.

Listing 1: Large richer CIO diagnostic experiment

```
/home/adrien-seigle/Desktop/CENTRALE/2A/Pole_Projet_S8/.venv/bin/python
iterative_cio_training.py \
--data-dir results/large_richer_cio \
--env-name mobile-large-central-v0 \
--iterations 10 \
--num-episodes 10 \
--max-steps 60 \
--epochs 10 \
--target failure_count \
--online-cio \
--online-cio-interval 10 \
--online-cio-lr 0.002 \
--cio-lr 0.02 \
--cio-clip 1.0 \
--fte-threshold 0.0 \
--ftl-threshold 0.1 \
--ping-pong-window 10 \
--ensure-bs-coverage \
--synthetic-neighbor-handovers 2 \
--device cpu
```

The most important outputs are `cio_training_progress.csv`, `cio_matrix.csv`, `cio_training_comprehensive.png`, `failure_distribution.png`, `cio_matrix_heatmap.png`, and `gym_simulation_handover_topology.png`.